Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Системы автоматизированного проектирования»

ОТЧЁТ
по лабораторной работе № 4

по дисциплине «Логика и основы алгоритмизации
в инженерных задачах»

Бинарное дерево поиска

Выполнили студенты группы 25BBB4:
Поликарпов Я., Егоров Е.

Приняли:
Юрова О. В., Деев М. В.

Пенза 2026

Цель и ход работы

Цель работы - изучить устройство бинарного дерева поиска, реализовать поиск введённого значения и подсчёт числа его вхождений, изменить добавление для исключения повторов и оценить сложность поиска.

В ходе работы мы разработали консольную программу на C++17 с системой сборки CMake. Из одного входного набора одновременно строятся два независимых дерева: с разрешёнными повторами и без повторов. Это позволяет проверить основные и дополнительные задания на одинаковых данных.

Структура данных и организация программы

Бинарное дерево состоит из узлов, каждый из которых хранит целое число и не более двух указателей на дочерние узлы. Корень хранится в указателе root_. Отсутствующая ветвь обозначается nullptr. Узел описан следующей структурой:

```
struct Node {
    int data;
    Node* left;
    Node* right;
};
```

В левом поддереве находятся значения, строго меньшие значения узла. В правом поддереве располагаются большие значения, а при разрешении повторов - также равные. Порядок ветвления при добавлении согласован с текстовым определением дерева поиска из задания.

Класс BinarySearchTree управляет выделением и освобождением памяти. Политика DuplicatePolicy::Allow разрешает повторное добавление, а DuplicatePolicy::Reject запрещает его. Основные методы класса: insert, contains, count, print, clear и size. Объявления размещены в binary_search_tree.hpp, реализация - в binary_search_tree.cpp, меню - в main.cpp.

Ввод и управление программой

Начальное дерево создаётся из чисел, вводимых через пробел или перевод строки. Значение -1 завершает этот этап и не добавляется. Затем меню позволяет вывести деревья, найти значение, подсчитать вхождения, добавить число, очистить деревья или завершить работу. Через пункт добавления можно сохранить и значение -1.

Ошибочные токены и числа вне диапазона int отклоняются с повторным запросом. При достижении конца входного потока программа завершается. Память освобождается при очистке дерева и в деструкторе; удаление узлов выполняется после удаления их поддеревьев.

Задание 1 Поиск значения в дереве

Построение контрольного дерева

Для пояснения алгоритмов использован набор 8, 3, 10, 1, 6, 14, 4, 7, 13, 6, 6. После его обработки обычное дерево содержит 11 узлов, дерево без повторов - 9. На рисунке 1 показан второй вариант; форма определяется порядком добавления чисел.

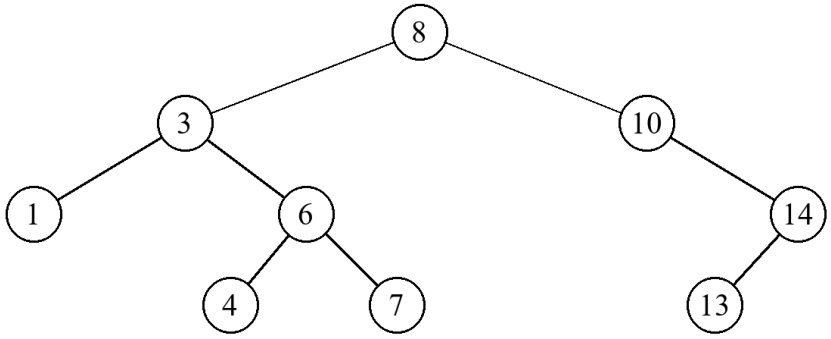


Рисунок 1 - Дерево без повторов для контрольного набора

Алгоритм поиска

Поиск начинается с корня. При равенстве функция возвращает true. Если искомое значение меньше значения текущего узла, рассматривается левое поддерево, иначе - правое. Пустой указатель означает, что значение отсутствует. Обход второго поддерева не требуется, поскольку его значения не могут совпасть с искомым.

```
bool BinarySearchTree::contains(int value) const noexcept {
    const Node* current = root_;
    while (current != nullptr) {
        if (value == current->data) {
            return true;
        }
        current = value < current->data ? current->left : current->right;
    }
    return false;
}
```

Например, поиск числа 7 проходит через узлы 8, 3, 6 и 7 и завершается успешно. Для числа 9 путь проходит через 8 и 10, после чего достигается пустая левая ветвь узла 10. Программа выводит сообщение об отсутствии значения.

На экран дерево выводится повёрнутым на 90 градусов: сначала правое поддерево, затем текущий узел с отступом и левое поддерево. Поэтому правые ветви находятся выше корня, а отступ отражает глубину узла.

Задание 2 Подсчёт числа вхождений

Метод count возвращает число узлов с заданным значением. Он использует свойство дерева поиска и не обходит заведомо неподходящие ветви. При значении, меньшем текущего, поиск продолжается слева. При равенстве увеличивается счётчик и просматривается правая ветвь; при большем значении также выбирается правая ветвь.

```
std::size_t BinarySearchTree::count(int value) const noexcept {
    std::size_t occurrences = 0;
    const Node* current = root_;
    while (current != nullptr) {
        if (value < current->data) {
            current = current->left;
        } else {
            if (value == current->data) {
                ++occurrences;
            }
            current = current->right;
        }
    }
    return occurrences;
}
```

После совпадения поиск продолжается, поскольку справа могут находиться другие вхождения. Они не обязаны быть непосредственными правыми потомками одинаковых узлов: между ними могут располагаться большие значения. При следующем сравнении алгоритм при необходимости снова переходит влево.

Для контрольного набора поиск всех вхождений числа 6 идёт по пути 8, 3, 6, 7, 6, 6. Счётчик увеличивается три раза. В дереве без повторов то же значение встречается один раз. Результаты приведены в таблице 1.

Таблица 1 - Подсчёт вхождений в контрольном наборе

Значение	С повторами	Без повторов
6	3	1
8	1	1
9	0	0
-5	0	0

Для пустого дерева и отсутствующего значения результат равен нулю. В дереве без повторов результат всегда равен нулю или единице. Подсчёт, как и поиск, выполнен итеративно и не требует стека рекурсивных вызовов.

Дополнительные задания

Пункт 1 Исключение повторного добавления

В функции insert перед переходом к следующей ветви проверяются равенство ключей и выбранная политика. Если значение уже найдено и установлена политика Reject, функция возвращает false. Новый узел не создаётся, размер дерева не меняется. В режиме Allow равное значение направляется в правое поддерево.

```
if (value == (*link)->data &&
    policy_ == DuplicatePolicy::Reject) {
    return false;
}
```

Указатель link имеет тип Node** и указывает на изменяемую связь: корень либо поле left или right текущего узла. После достижения пустой связи создаётся Node, а счётчик узлов увеличивается. Отдельный предварительный поиск не нужен: проверка повторов выполняется в том же проходе.

Пункт 2 Оценка сложности поиска

Обозначим через n число узлов, через h - число узлов на самом длинном пути от корня до листа. На каждом шаге поиска остаётся только одно поддерево. Поэтому число посещённых узлов не превышает h, время равно O(h), а дополнительная память итеративной реализации - O(1).

Таблица 2 - Сложность основных операций

Операция	Время	Дополнительная память
Поиск contains	O(h)	O(1)
Подсчёт count	O(h)	O(1)
Добавление insert	O(h + 1)	O(1)*
Получение размера size	O(1)	O(1)
Очистка clear	O(n)	O(h)

* Для добавления указана память без учёта нового узла. Само дерево занимает O(n) памяти. Очистка рекурсивна, поэтому использует O(h) памяти стека.

Лучший случай успешного поиска - совпадение в корне, O(1). Для сбалансированного дерева h = O(log n), поэтому поиск имеет сложность O(log n). Например, в идеально сбалансированном дереве из 1023 узлов посещается не более 10 узлов.

В худшем случае дерево вырождается в цепочку, h = n и поиск требует O(n) времени. Такая форма возникает при последовательном добавлении возрастающих или убывающих ключей; повторы также могут образовывать цепочку. Для случайного порядка различных ключей ожидаемая сложность - O(log n). Балансировка в данной программе не выполняется, поэтому эта оценка не гарантируется для любого ввода.

Результаты проверки программы

Проверка выполнена под Windows в сборке Release компилятором GCC 15.2.0 из комплекта CLion. Для основного сценария введены следующие 35 чисел; затем введён признак завершения -1:

```
2 3 4 5 68 55 58 437 7347 74 347 7578 58 5 34 346
3634 47 75 5 57457 75 457 574 574574745 457 574
745457 5 8 6858 434 34 346 347 -1
```

В наборе 25 различных значений. На нём проверены поиск, подсчёт и различие политик добавления. После начального построения последовательно добавлены числа 3 и 31132424. Результаты совпали с ожидаемыми.

Таблица 3 - Результаты контрольного запуска

Проверка	Значение	С повторами	Без повторов
Начальное число узлов	-	35	25
Поиск отсутствующего	12	Не найдено	Не найдено
Поиск имеющегося	3	Найдено	Найдено
Подсчёт до добавлений	5	4	1
Повторное добавление	3	Добавлено	Отклонено
Добавление нового	31132424	Добавлено	Добавлено
Итоговое число узлов	-	37	26
Подсчёт после добавлений	3	2	1

При вводе токена 34к4 программа сообщает об ошибке, поскольку токен содержит букву. Число 25334645775 также отклоняется: оно превышает верхнюю границу 32-разрядного int, равную 2147483647. После корректного ввода 31132424 работа продолжается; ошибочные значения не изменяют деревья.

Сборка

Проект собирается следующими командами; для Windows с генератором Ninja или MinGW программа запускается как build\lab4.exe:

```
cmake -S . -B build
cmake --build build --config Release
```

Псевдокод основных алгоритмов

Добавление элемента

```
Insert(value, policy):
  link := адрес root
  пока *link != null:
    если value = (*link).data И policy = Reject:
      вернуть false
    если value < (*link).data:
      link := адрес (*link).left
    иначе:
      link := адрес (*link).right
  *link := новый Node(value, null, null)
  size := size + 1
  вернуть true
```

Поиск значения

```
Contains(value):
  current := root
  пока current != null:
    если value = current.data: вернуть true
    если value < current.data: current := current.left
    иначе: current := current.right
  вернуть false
```

Подсчёт вхождений

```
Count(value):
  result := 0; current := root
  пока current != null:
    если value < current.data:
      current := current.left
    иначе:
      если value = current.data: result := result + 1
      current := current.right
  вернуть result
```

Освобождение памяти

```
Destroy(node):
  если node = null: вернуть
  Destroy(node.left)
  Destroy(node.right)
  удалить node
```

При полной очистке после Destroy(root) корневой указатель обнуляется, размер устанавливается в ноль. Деструктор дерева вызывает ту же процедуру очистки.

Вывод

В ходе лабораторной работы мы изучили бинарное дерево поиска и реализовали его на C++17 с использованием указателей. Разработали добавление узлов, поиск вводимого значения, подсчёт всех вхождений, вывод структуры и освобождение памяти. Для дополнительного задания изменили добавление: при политике Reject равное значение не создаёт новый узел. Построение двух деревьев на одном наборе позволило непосредственно сравнить результаты.

Контрольный запуск подтвердил корректность поиска, обработки ошибочного ввода и работы с повторами: исходные 35 значений дали 35 узлов в обычном дереве и 25 в дереве без повторов. После добавления повторной тройки и нового значения размеры стали равны 37 и 26. Подсчёт числа 5 дал 4 и 1, а числа 3 после добавления - 2 и 1. Две группы автоматических проверок CTest завершились успешно.

Установили, что время поиска определяется высотой дерева и составляет O(h). Для сбалансированной формы это O(log n), а при вырождении в цепочку - O(n). Итеративные поиск и подсчёт используют O(1) дополнительной памяти. Следовательно, дерево поиска удобно для упорядоченного хранения данных, однако порядок добавления и наличие повторов существенно влияют на его высоту и скорость операций.